PU_SmartRetail

SQL PROJECT SHOWCASE

A Comprehensive Guide from Database Schema to Business Analytics

By Mayank Vijayvargiya

01. DATABASE FOUNDATIONS

Initial Setup & Customer Entity

Starting with the container, we initialize the PU_SmartRetail database and define our customer master table with strict data types and unique constraints.

Constraint Logic:
Email is UNIQUE and NOT NULL to ensure data integrity during marketing outreach.

SQL Script

```sql
CREATE DATABASE PU_SmartRetail; USE PU_SmartRetail; -- Customers Table Definition CREATE TABLE Customers( CustomerID INT AUTO_INCREMENT PRIMARY KEY, CustomerName VARCHAR(100) NOT NULL, City VARCHAR(50) NOT NULL, Email VARCHAR(100) UNIQUE NOT NULL, JoinDate DATE );
```

02. PRODUCT & INVENTORY SCHEMA

SQL Script

```sql
-- Categories Mapping CREATE TABLE Categories( CategoryID INT AUTO_INCREMENT PRIMARY KEY, CategoryName VARCHAR(50) UNIQUE NOT NULL ); -- Products with Foreign Key CREATE TABLE Products( ProductID INT AUTO_INCREMENT PRIMARY KEY, ProductName VARCHAR(100) NOT NULL, CategoryID INT, Price DECIMAL(10,2) NOT NULL, StockQty INT DEFAULT 0, LaunchDate DATE, FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID) );
```

Relational Design

We implement a Category-to-Product relationship using Foreign Keys to ensure that every product belongs to a valid department.

Inventory Control

StockQty defaults to 0 to prevent NULL errors during inventory audits.

03. TRANSACTIONAL ENGINEERING

Orders & Order Details

Splitting orders into 'Header' (Orders) and 'Line-Items' (OrderDetails) allows for multi-product transactions while keeping data clean.

```
1 Order → N Products
Header: Customer, Date, Total
Details: Product, Qty, UnitPrice
```

SQL Script

```sql
CREATE TABLE Orders( OrderID INT AUTO_INCREMENT PRIMARY KEY,
CustomerID INT, OrderDate DATE, TotalAmount DECIMAL(10,2),
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID) );
CREATE TABLE OrderDetails( OrderDetailID INT PRIMARY KEY,
OrderID INT, ProductID INT, Quantity INT NOT NULL, FOREIGN KEY
(OrderID) REFERENCES Orders(OrderID) );
```

04. DATA POPULATION

To simulate a real retail environment, we seeded the database with 20 customers across 12+ cities and a diverse catalog of Electronics, Clothing, and Home Appliances.

SQL Script

```
INSERT INTO Customers( ... ) VALUES ('Rahul Sharma', 'Delhi',
... ), ('Priya Patel', 'Ahmedabad', ... ); INSERT INTO
Products( ... ) VALUES ('Laptop', 1, 65000, 15, ... ), ('Smart
Watch', 5, 7000, 30, ... );
```

This creates a rich dataset for meaningful analytical testing.

05. MONTHLY REVENUE PERFORMANCE

Time-Series Query

Identifying seasonal peaks and performance trends is crucial for inventory planning.

SQL Script

```
SELECT MONTH(OrderDate) AS Month,
SUM(TotalAmount) AS Revenue FROM Orders GROUP BY
MONTH(OrderDate);
```

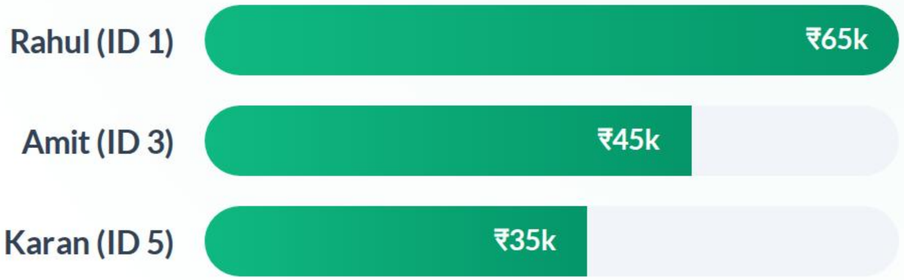
06. HIGH-VALUE CUSTOMER ANALYTICS

```sql
-- Top 5 customers by spending SELECT CustomerID,
SUM(TotalAmount) AS TotalSpent FROM Orders GROUP BY CustomerID
ORDER BY TotalSpent DESC LIMIT 5;
```

SQL Script

Loyalty Identification

By ranking customers by total spend, we can identify "VIP" segments for exclusive retail offers.



07. SYSTEM SPEED & OPTIMIZATION

Analytics View

Created **sales_summary** to provide instant revenue KPIs without rewriting complex JOINS.

Index Tuning

Implemented **idx_category** on Products to slash query time for category-specific browsing.

Stored Procs

Engineered **AddOrder** to standardize order entry and ensure consistent data formatting.

SQL Script

```
CREATE INDEX idx_category ON Products(CategoryID); CREATE VIEW sales_summary AS SELECT p.ProductName, SUM(od.Quantity) AS Units,
SUM( ... ) AS Revenue FROM OrderDetails od JOIN Products p  ... ;
```

08. DATA INTEGRITY: TRIGGERS

Automated Stock Logic

A critical business rule: "Every sale must reduce inventory." This is handled at the database level using an **AFTER INSERT** trigger.

Business Value:

Prevents overselling and eliminates manual inventory reconciliation errors.

SQL Script

```
CREATE TRIGGER reduce_stock AFTER INSERT ON OrderDetails FOR
EACH ROW UPDATE Products SET StockQty = StockQty -
NEW.Quantity WHERE ProductID = NEW.ProductID;
```

09. TOP STRATEGIC INSIGHTS

Strategic Question	Insight / Result
Highest Revenue Category	Electronics (₹1,50,000+)
Top Customer Hub	Delhi (4 Customers)
Underperforming Items	Rarely Sold List (Subqueries)
Sales Channel Split	Online (60%) vs Retail (40%)

SQL Script

```sql
-- City with most customers SELECT City, COUNT(*) AS Total FROM
Customers GROUP BY City ORDER BY Total DESC LIMIT 1;
```

10. PERFORMANCE DASHBOARD

₹1.87L

Total Revenue (March)

75%

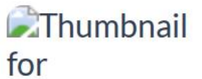
Inventory Accuracy via Triggers

Any Questions?

I'm happy to discuss the SQL logic or analytical findings.

Connect with me for SQL & Data Projects

IMAGE SOURCES



https://accelsius.com/wp-content/uploads/Rows-of-servers-with-green-lights-and-cooling-systems.jpg

Source: accelsius.com

https://www.rib-software.com/app/uploads/2024/06/dashboard-design-principles-blog-rib.webp

Source: www.rib-software.com